

Module - 1

Introduction to Algo Analysis

- characteristics of Algorithm.
- Criteria for Analysing Algos.
- Time and Space Complexity
 - Best Case
 - Worst case
 - Average case.
- Asymptotic Notations
 - Big Oh (O)
 - Big Omega (Ω)
 - Big Theta (Θ)
 - Little oh (o), and
 - Little omega (ω)
- classifying functions by their asymptotic growth rate.
- Complexity Calculation of Simple algorithms.

- Analysis of Recursive Algorithms.
- Recurrence Equations.
- Solving Recurrence Equations
 - Iteration method.
(Substitution method)
 - Recursion tree method.
 - ~~and~~ Master's Theorem.

MODULE I

(5)

INTRODUCTION TO ALGM ANALYSIS

ALGORITHM

An Algorithm is any well defined computational procedure that takes some value as input and produces some value as output.

An algorithm is thus a sequence of computational steps that transform the input into the output.

Or it is a step by step procedure to solve a problem.

That is, an algorithm is a tool for solving a well specified computational problem.

All algorithms must satisfy the following criteria:

1. Input

Zero or more quantities externally supplied.

2. Output

At least one quantity is produced.

3. Definiteness

Each instruction must be clear and unambiguous.

4. Finiteness

Algorithm must terminate after a finite number steps.

5. Effectiveness

Every instruction must be very basic, so that it can be carried out, i.e., by a person using a pen and paper.

An algorithm is said to be correct if for every instances of input it halts with correct output.

ALGORITHM ANALYSIS

Once an algorithm is given for a problem and decided to be correct, an important step is to determine the efficiency of the algorithm.

Determining algorithm efficiency is known as analysis of algorithm.

The process of algorithm analysis refers to defining how much time and memory is utilized in solving a problem. These are respectively known as

- Time complexity
- Space Complexity.

Time Complexity

Amount of an algorithm needs to run to its completion.

There are two practical approaches to estimate the running time.

They are

- Operation Count
- Step Count.

Operation Count

Identify one or more key operations such as addition or multiplication, then compare and determine the number of times these are performed.

Success of this method depends on the ability to identify operations that contribute most of the complexity.

Example :- Sum of n numbers

```
Sum(a, n)
```

```
Sum  $\leftarrow$  0
```

```
for  $i \leftarrow 1$  to  $n$ , do
```

```
Sum  $\leftarrow$  Sum +  $a[i]$ 
```

```
return Sum.
```

Here the elementary operation that contributes complexity is

Sum $\leftarrow$ Sum + $a[i]$

This instruction executes 'n' times in this program.

So the time complexity is 'n' and it can be represented as $O(n)$.

Step Count

Determine the total number of steps executed by the program.

Apart from the operation count method, in step count method, we attempt to count for the time spend in all part of the program.

Example :- Sum of 'n' numbers

Sum(a, n)	
Sum $\leftarrow$ 0	$\Rightarrow 1$
for i $\leftarrow$ 1 to n, do	$\Rightarrow n+1$
Sum $\leftarrow$ Sum + a[i]	$\Rightarrow n$
return Sum.	$\Rightarrow 1$
	$2n+3$

Total running time $2n+3$ and we can represent it as $O(n)$.

Types of Analysis - Best, Worst and Average Case Complexity

An algorithm may have different behaviours of different classes of input of fixed size.

The best, worst and average cases of a given algorithm express the resource usage is "atleast", "atmost" and "on average" respectively.

Usually the resource being considered is running time but it could also be memory or other resource.

The best case gives the minimum time, the worst case running time gives the maximum time, the ^{average} ~~best~~ case running time gives the time required on average to execute the algorithm.

Worst Case

Worst case complexity of the algorithm is the function defined by the maximum number of steps taken on any instance of size n .

Worst case running time of an algorithm is an upper bound on the running time for any input.

An algorithm will never take any longer time than this

Average Case

Average case complexity of the algorithm is the function defined by the average number of steps taken on any instance of size n .

The average case running time of an algorithm is an estimate of the running time for an "average" input.

It is expected behaviour of the algorithm when the input

is randomly drawn from a given distribution.

Best case

Best case complexity of the algorithm is the function defined by the minimum no. of steps taken on any instance of size n .

Recurrence Equations

When an algorithm contains a recursive call to itself, its running time can often be described by a recurrence.

A recurrence is an equation or inequality that describes a function in terms of its value on smaller inputs.

For example, the worst case running time $T(n)$ of merge sort

procedure could be described by (3)
the recurrence

$$T(n) = \begin{cases} O(1) & \text{if } n=1 \\ 2T(n/2) + O(n) & \text{if } n>1 \end{cases}$$

Mainly four methods are used for solving recurrence. They are.

- Iteration method
- Recursion tree method.
- Master's theorem, and
- Substitution method.

Iteration Method.

This method also known as "Try backsubstitute until you know what is going on".

That is in this method, we iteratively apply the recurrence equation to itself until we find a pattern.

In this method, the basic idea is to expand the recurrence and express it as a summation.

of terms dependent only on n and its initial conditions.

6 And solve the general eqn or summation equation.

Example

$$T(n) = 2T(n/2) + bn \quad \text{--- (1)}$$

Soln

$$T(n/2) = 2T(n/4) + bn/2$$

Apply $T(n/2)$ in (1)

$$T(n) = 2[2T(n/4) + bn/2] + bn$$

$$T(n) = 4T(n/4) + 2bn \quad \text{--- (2)}$$
$$= 2^2 T(n/2^2) + 2bn.$$

$$T(n/4) = 2T(n/8) + bn/4$$

Apply $T(n/4)$ in (2)

$$T(n) = 4[2T(n/8) + bn/4] + 2bn$$

$$T(n) = 8T(n/8) + 3bn \quad \text{--- (3)}$$

$$= 2^3 T(n/2^3) + 3bn.$$

∴ General term

$$T(n) = 2^i T(n/2^i) + ibn.$$

This series will end when

$$T(n/2^i) = T(1)$$

$$\text{i.e., } n/2^i = 1$$

$$n = 2^i$$

$$\log n = i \log 2$$

$$i = \frac{\log n}{\log 2}$$

$$= \log_2 n = \lceil \log_2 n \rceil$$

Apply these values in the General equation

$$T(n) = 2^i T(n/2^i) + i b n$$

$$= n T(n/n) + b n \log_2 n$$

$$= n T(1) + b n \log_2 n$$

$$= n + b n \log n$$

$$\text{put } b = 1$$

$$T(n) = n + n \log n$$

When giving values to n , we can

see that $n \log n$ is fastest growing

fn. So $T(n) = O(n \log n)$

Q. Solve the recurrence eqⁿ

$$T(n) = \begin{cases} 2 & n=0 \\ 2 + T(n-1) & \text{when } n > 0 \end{cases}$$

using iteration method.

Ans:

$$T(n) = T(n-1) + 2 \quad \text{--- (1)}$$

$$T(n-1) = T(n-2) + 2$$

Sub in (1)

$$T(n) = T(n-2) + 2 + 2$$

$$T(n) = T(n-2) + 2 + 2 \quad \text{--- (2)}$$

$$T(n-2) = T(n-3) + 2$$

Sub in (2)

$$T(n) = T(n-3) + 2 + 2 + 2$$

$$T(n) = T(n-3) + 2 + 2 + 2 \quad \text{--- (3)}$$

General eqⁿ

$$T(n) = T(n-i) + 2 + 2 + 2 + \dots$$

$$\text{Let } n-i = 0 \\ n = i$$

$$\begin{aligned}
 \Rightarrow T(n) &= T(n) + 2 \\
 &= 2 + 2n \\
 &= \underline{\underline{O(n)}}
 \end{aligned}$$

$$Q \Rightarrow T(n) = T(n-1) + n \quad \text{--- (1)}$$

$$T(n-1) = T(n-2) + n-1$$

Sub in (1)

$$T(n) = T(n-2) + n-1 + n \quad \text{--- (2)}$$

$$T(n-2) = T(n-3) + n-2$$

Sub in (2)

$$T(n) = T(n-3) + n-2 + n-1 + n \quad \text{--- (3)}$$

$$\text{General eqn} = T(n-i) + \frac{n(n+1)}{2}$$

$$n-i = 1, \quad n=i$$

$$\begin{aligned}
 \Rightarrow T(n) &= T(1) + \frac{n(n+1)}{2} \\
 &= \underline{\underline{O(n^2)}}
 \end{aligned}$$

Recursion tree Method

Recursion trees are useful when the recurrence describe the summing time of a divide and conquer problem.

This method converts the recurrence into a tree, where node represents the cost of a single subproblem.

Then we sum the costs within each level of the tree to obtain a set of per-level costs.

And then we sum all the per level costs to determine the total cost of all levels of the recursion.

Then we solve the summation equation.

Example - 1

$$T(n) = T(n-1) + n.$$

$$\begin{array}{ccc} n & \rightarrow & n \\ | & & \\ n-1 & \rightarrow & n-1 \\ | & & \\ n-2 & \rightarrow & n-2 \\ | & & \\ n-3 & \rightarrow & n-3 \\ \vdots & & \\ T(1) & \rightarrow & n-i \end{array}$$

Summation Eqn

$$T(n) = n + n-1 + n-2 + \dots + B.C.$$

Boundary Condo. B.C.

$$n-i = 1$$

$$\begin{aligned} \text{max } i &= n-1 \\ &= O(n) \end{aligned}$$

Apply in Summation Eqn.

$$T(n) = n + n-1 + n-2 + \dots + O(n).$$

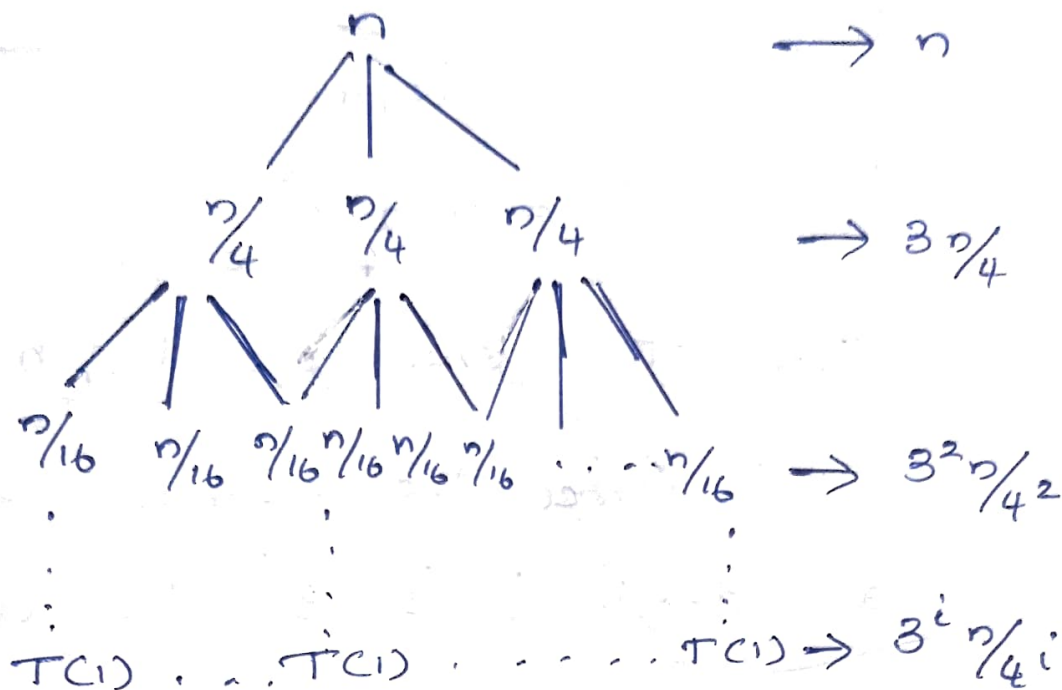
$$= \frac{n(n+1)}{2} + O(n)$$

$$= \frac{n^2 + n}{2} + O(n)$$

$$= \underline{\underline{O(n^2)}}$$

Example-2:

$$T(n) = 3T(n/4) + n$$



Subproblem size decreases as we get further from the root.

We eventually reach a boundary condition.

It happens when the size of subproblem for a node at depth i hits 1.

$$\text{i.e., } T(n/4^i) = T(1)$$

$$n/4^i = 1$$

$$n = 4^i$$

$$\log n = i \log 4$$

$$i = \frac{\log n}{\log 4}$$

$$i = \log_4 n$$

Thus the tree has $\log_4 n + 1$ levels.

Since the number of nodes at depth i is 3^i , the total cost over all nodes at depth i , is

$$3^i \cdot n/4^i = 3^{\log_4 n} \quad [n/4^i = 1]$$

$$= n^{\log_4 3}.$$

So the last level, at depth $\log_4 n$, has $n^{\log_4 3}$ nodes, each having cost $\Theta(1)$, for a total cost $\Theta(n^{\log_4 3})$.

Next we add up the costs over all levels to determine the cost

for the entire tree.

(45)

$$T(n) = n + 3n/4 + 3^2n/4^2 + \dots + (3/4)^{\log_3 n} n + O(n^{\log_3 3}).$$

$$= \sum_{i=0}^{\log_3 n - 1} (3/4)^i n + O(n^{\log_3 3}).$$

$$= \sum_{i=0}^{\infty} (3/4)^i n + O(n^{\log_3 3}) \text{ --- (1)}$$

According to the equation

$$\sum_{k=0}^{\infty} r^k = \frac{1}{1-r}, \text{ we can}$$

rewrite the equation (1) as below

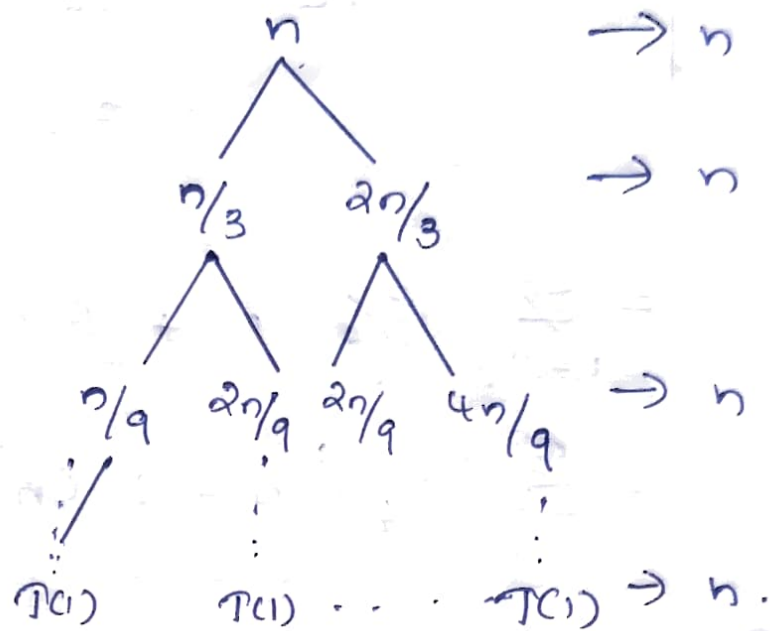
$$= \frac{1}{1-3/4} n + O(n^{\log_3 3})$$

$$= 4n + O(n^{\log_3 3})$$

$$= \underline{\underline{O(n)}}$$

Example - 3:

$$T(n) = T(n/3) + T(2n/3) + n.$$



Here we take the largest path

$$n \rightarrow 2n/3 \rightarrow (2/3)^2 n \rightarrow \dots \rightarrow (2/3)^i n$$

Boundary Condition occurs when

$$(2/3)^i n = 1$$

$$(2/3)^i = 1/n$$

$$i \log 2/3 = \log 1/n$$

$$i = \frac{\log 1/n}{\log 2/3}$$

$$= \frac{-\log n}{-\log 3/2}$$

$$i = \log_{3/2} n$$

i.e, the tree has $\log_{3/2} n + 1$ levels.

$$= \left[\text{height} = \log_{3/2} n \right]$$

Summation Equation

$$\begin{aligned} T(n) &= n + n + n + \dots + n + O(\log_{3/2} n) \\ &= \sum_{i=0}^{\log_{3/2} n - 1} n + O(\log_{3/2} n) \end{aligned}$$

Here the solution to the recurrence will be at most the number of levels times the cost of each level

$$T(n) \Rightarrow O(n \log n) \Rightarrow \text{Worst case.}$$

$$\underline{\underline{T(n) \Rightarrow \Omega(n \log n) \Rightarrow \text{Best case.}}}$$

Master's Method.

The master's method provides bounds for solving recurrences of the form $T(n) = aT(n/b) + f(n)$, where $a \geq 1$ and $b > 1$ are constants and $f(n)$ is an asymptotically positive function.

Here "a" represents "No. of subproblems" and "b" represents "size of subproblem".

$T(n)$ can be bounded asymptotically as follows:-

case 1: $f(n) < n^{\log_b a}$

$\Rightarrow f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then

$$T(n) = O(n^{\log_b a})$$

6

Case 2: $f(n) = n^{\log_b a}$

$\Rightarrow f(n) = \Theta(n^{\log_b a})$, then.

$T(n) = \Theta(n^{\log_b a} \log n)$

Case 3: $f(n) > n^{\log_b a}$

$\Rightarrow f(n) = \Omega(n^{\log_b a + \epsilon})$ for

some constant $\epsilon > 0$ and if

$a f(n/b) \leq c \cdot f(n)$ for some

constant $c < 1$, then

$T(n) = \Theta(f(n))$

Master's Theorem

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = a T(n/b) + f(n),$$

where we interpret n/b to mean either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$.

Then $T(n)$ can be bounded asymptotically as follows:

① If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = O(n^{\log_b a})$

② If $f(n) = O(n^{\log_b a})$, then
 $T(n) = O(n^{\log_b a} \log n)$

③ If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $a_f(n/b) \leq c f(n)$ for some constant $c < 1$, and all sufficiently large n , then
 $T(n) = O(f(n))$

Example 1

$$T(n) = 4T(n/2) + n$$

$$a = 4 \quad b = 2 \quad f(n) = n$$

$$n^{\log_b a} = n^{\log_2 4} = n^2$$

$$f(n) < n^{\log_b a}$$

or $\therefore$ Case 1

$$\begin{aligned} f(n) &= O(n^{\log_b a - \epsilon}) \\ &= O(n^{2 - \epsilon}) \\ &= O(n^{2-1}) \\ &= O(n) \end{aligned}$$

$$\log_b a = \frac{\log a}{\log b}$$

$$\log_2 4 = \frac{\log 4}{\log 2} = 2$$

$$T(n) = O(n^{\log_b a})$$

$$= O(n^2)$$

8

Example-2

$$T(n) = 4T(n/2) + n^2$$

$$a = 4 \quad b = 2 \quad f(n) = n^2$$

$$n^{\log_b a} = n^{\log_2 4} = n^2$$

$$f(n) = n^2$$

$\therefore$ Case 2

$$\Rightarrow f(n) = O(n^{\log_b a})$$

$$= O(n^2)$$

$$T(n) = O(n^{\log_b a} \log n)$$

$$= O(n^2 \log n)$$

Example-3

$$T(n) = 4T(n/2) + n^3$$

$$a = 4 \quad b = 2 \quad f(n) = n^3$$

$$n^{\log_b a} = n^{\log_2 4} = n^2$$

$$f(n) > n \log_b a$$

9

$\therefore$ case 3

$$\Rightarrow f(n) = \Omega(n \log_b a + \epsilon)$$

$$= \Omega(n^{2+\epsilon})$$

$$= \Omega(n^{2+1})$$

$$= \Omega(n^3)$$

Here we have to check

$$a f(n/b) \leq c \cdot f(n), \quad c < 1$$

$$4 \cdot f(n/2) \leq c \cdot n^3$$

$$4 \cdot (n/2)^3 \leq c n^3$$

$$4 \cdot \frac{n^3}{8} \leq c n^3$$

$$\frac{n^3}{2} \leq c n^3$$

$$\frac{1}{2} \leq c$$

$\therefore c < 1$ therefore condition
satisfied

$$\therefore T(n) = \Theta(f(n))$$

$$= \underline{\underline{\Theta(n^3)}}$$

Master's Theorem

$$T(n) = aT(n/b) + \Theta(n^k \log^p n)$$

$a \geq 1$ $b \geq 1$ $k \geq 0$ and P is a real no.

1 $\Rightarrow$ if $a > b^k$, then

$$T(n) = \Theta(n^{\log_b a})$$

2 $\Rightarrow$ if $a = b^k$

(a) $\Rightarrow$ if $p > -1$, then

$$T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$$

(b) $\Rightarrow$ if $p = -1$, then

$$T(n) = \Theta(n^{\log_b a} \log(\log n))$$

(c) $\Rightarrow$ if $p < -1$, then

$$T(n) = \Theta(n^{\log_b a})$$

3 $\Rightarrow$ if $a < b^k$

(a) $\Rightarrow$ if $p \geq 0$, then

$$T(n) = \Theta(n^k \log^p n)$$

(b) $\Rightarrow$ if $p < 0$, then

$$T(n) = \Theta(n^k)$$

Example - 1

$$T(n) = 2T(n/2) + n \log n$$

$$a = 2, b = 2, k = 1, p = 1$$

$$b^k = 2^1 = 2$$

$$a = b^k \text{ and } p > -1$$

$$T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$$

$$n^{\log_b a} = n^{\log_2 2} = n$$

$$\log^{p+1} = \log^{1+1} = \log^2$$

$$\underline{T(n) = \Theta(n \log^2 n)}$$

Example 2

$$T(n) = 3T(n/4) + n \log n$$

$$a = 3, b = 4, k = 1, p = 1$$

$$b^k = 4^1 = 4$$

$$a < b^k, p \geq 0$$

$$T(n) = \Theta(n^k \log^p n)$$

$$= \underline{\underline{\Theta(n \log^2 n)}}$$

Example 3

$$T(n) = T(n/2) + n^2$$

$$a = 1, b = 2, k = 2, p = 0$$

$$b^k = 2^2 = 4$$

$$a < b^k$$

$$T(n) = \Theta(n^k \log^p n)$$

$$= \Theta(n^2 \log^0 n)$$

$$= \underline{\underline{\Theta(n^2)}}$$

Asymptotic Analysis

It is the easiest way to describe the running time of an algorithm.

It is used to indicate the relative growth of functions as n gets very large.

Asymptotic Notation

It is used to describe the asymptotic running time of an algorithm.

They are

- Big oh $[O]$
- Big omega $[\Omega]$
- Big theta $[\Theta]$
- Little oh $[o]$, and
- Little omega $[ω]$.

Big Oh Notation [O]

11

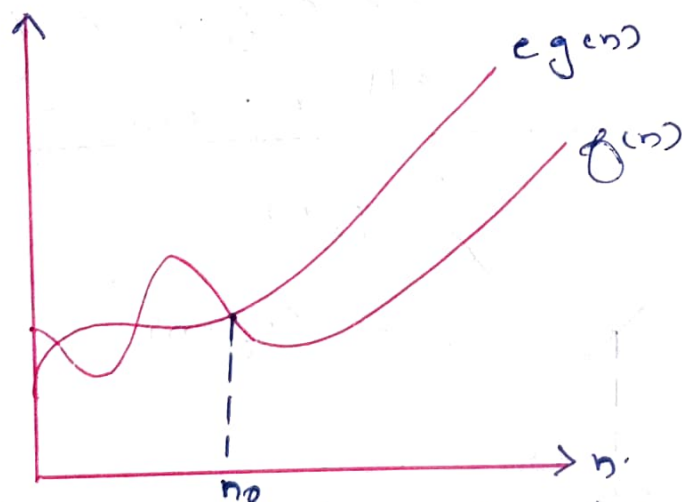
Asymptotic Upper Bound

Used to indicate the maximum number of operations an algorithm can perform.

Definition

For a given function $g(n)$, we denote $O(g(n))$, the set of functions

$$O(g(n)) = \{ f(n) : \text{there exist two constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0. \}$$



For all values n to the right of n_0 , the value of the function $f(n)$ is on or below $g(n)$.

Big Omega Notation [Ω]

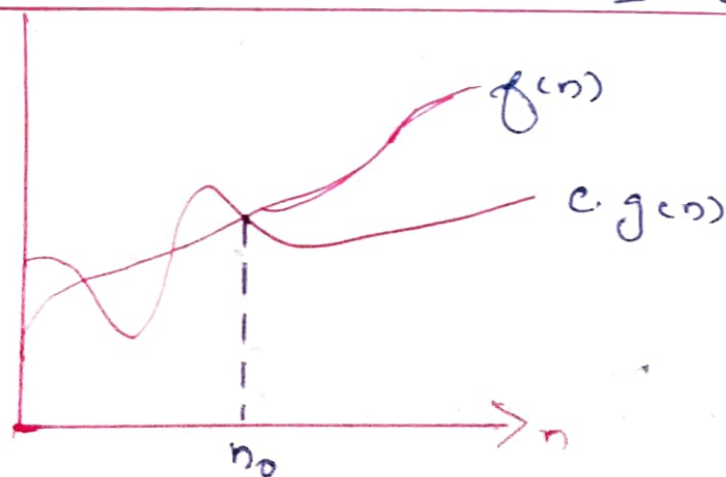
Asymptotic Lower Bound.

Used to indicate the minimum number of operations an algorithm can perform.

Definition.

For a given function $g(n)$, we denote by $\Omega(g(n))$, the set of functions,

$$\Omega(g(n)) = \{f(n), \text{ if there exist two constants 'c' and 'n}_0\text{ such that } 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0. \}$$



for all values n , to the right

of n_0 , the value of $f(n)$ is on or above $c \cdot g(n)$ (13)

Big theta Notation Θ

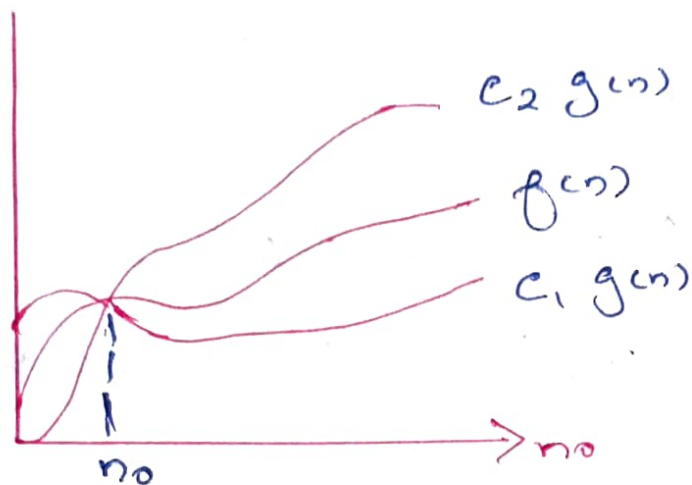
Asymptotic Tightest Bound.

Definition

For a given function $g(n)$ we denote by $\Theta(g(n))$, the set of functions

$$\Theta(g(n)) = \{f(n) : \text{there exist two constants } c_1, c_2, \text{ and } n_0 \text{ such that}$$
$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$$
$$\text{for all } n \geq n_0.\}$$

A function $f(n)$ belongs to the set $\Theta(g(n))$ if there exist two constants c_1 and c_2 such that it can be in between $c_1(g(n))$ and $c_2(g(n))$ for sufficiently large n .



For all values of 'n' to the right of n_0 , the value of $f(n)$ lies at or above $c_1(g(n))$ and at or below $c_2(g(n))$

Theorem

For any two functions $f(n)$ and $g(n)$, we have $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Proof

$f(n) = \Omega(g(n))$ if

$$0 \leq c_1 g(n) \leq f(n)$$

$f(n) = O(g(n))$ if

(15)

$$0 \leq f(n) \leq c_2 g(n)$$

$$f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n))$$

$$\Rightarrow c_1 g(n) \leq f(n) \leq c_2 g(n)$$

$$\Rightarrow f(n) = \Theta(g(n))$$

Hence Proved.

Little oh [o]

The asymptotic upper bound provided by O -notation [Big oh] may or may not be asymptotically tight.

i.e., The bound $2n^2 = O(n^2)$ is asymptotically tight but the bound $2n^2 = O(n^3)$ is not.

We use o -notation [Little oh] to denote an upper bound that is not asymptotically tight.

Definition

For given function $g(n)$, we

denote by $O(g(n))$, the set of functions

$$O(g(n)) = \{f(n) : \text{for any true constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0.\}$$

Theorem

$f(n) = O(g(n))$ if and only if

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0.$$

Example 1

$$f(n) = 3n + 10 = O(n^2) \quad ?$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{3n + 10}{n^2}$$

$$= \lim_{n \rightarrow \infty} \left[\frac{3}{n} + \frac{10}{n^2} \right]$$

$$= \frac{3}{\infty} + \frac{10}{\infty}$$

$$\frac{a}{\infty} = 0$$

$$= 0$$

$$\therefore 3n + 10 = O(n^2)$$

Example 2

(17)

$$f(n) = 3n + 10 = o(n)?$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{3n + 10}{n}$$

$$= \lim_{n \rightarrow \infty} \left[3 + \frac{10}{n} \right]$$

$$= 3 + \frac{10}{\infty}$$

$$= 3 \neq 0$$

$$\therefore \underline{3n + 10 \neq o(n)}.$$

Little Omega ω

We use ω notation to denote a lower bound that is not asymptotically tight.

Definition

For a given function $g(n)$, we denote by $\omega(g(n))$, the set of functions

$$\omega(g(n)) = \{ f(n) : \text{for any true constants } c > 0, \text{ there exist a constant } n_0 > 0, \text{ such} \}$$

that $0 \leq c \cdot g(n) < f(n)$
for all $n \geq n_0$ }

Theorem

$f(n) = \omega(g(n))$ if and only if

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

Example 1

Is $3n^2 + 10 = \omega(n)$?

$$f(n) = 3n^2 + 10 \quad g(n) = n$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{3n^2 + 10}{n}$$

$$= \lim_{n \rightarrow \infty} \left[3n + \frac{10}{n} \right]$$

$$= \left[3 \cdot \infty + \frac{10}{\infty} \right]$$

$$\boxed{a \cdot \infty = \infty}$$

$$= \infty$$

$$\therefore 3n^2 + 10 = \omega(n)$$

Example 2

Is $3n+10 = \omega(n)$?

$$f(n) = 3n+10 \quad g(n) = n$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{3n+10}{n}$$

$$= \lim_{n \rightarrow \infty} \left[3 + \frac{10}{n} \right]$$

$$= [3 + 10/0]$$

$$= 3 \neq \infty$$

$$\therefore 3n+10 \neq \omega(n)$$

Applications of Asymptotic Notations in Algorithm Analysis

We discuss algorithms with respect to problems.

If you have an algorithm for a problem, and yours has better time complexity, we say that algorithm is more efficient with

respect to the problem.

The order of growth of the running time of an algorithm, gives a simple characterization of the algorithm's efficiency and also allows us to compare the relative performance of alternative algorithms.

Problems are comprised of problem instances.

There are typically infinitely.

So, if you want to capture the complexity of all the instances of the problem, with respect to the input size, we are studying the asymptotic efficiency of algs.

Properties - Asymptotic Notation

Transitivity

$$\Rightarrow f(n) = O(g(n)) \text{ and } g(n) = O(h(n))$$

$$\Rightarrow f(n) = O(h(n))$$

$$\Rightarrow f(n) = \Omega(g(n)) \text{ and } g(n) = \Omega(h(n))$$

$$\Rightarrow f(n) = \Omega(h(n))$$

$$\Rightarrow f(n) = \Theta(g(n)) \text{ and } g(n) = \Theta(h(n))$$

$$\Rightarrow f(n) = \Theta(h(n))$$

$$\Rightarrow f(n) = o(g(n)) \text{ and } g(n) = o(h(n))$$

$$\Rightarrow f(n) = o(h(n))$$

$$\Rightarrow f(n) = \omega(g(n)) \text{ and } g(n) = \omega(h(n))$$

$$\Rightarrow f(n) = \omega(h(n))$$

Reflexivity

$$f(n) = O(f(n))$$

$$f(n) = \Omega(f(n))$$

$$f(n) = \Theta(f(n))$$

Symmetry

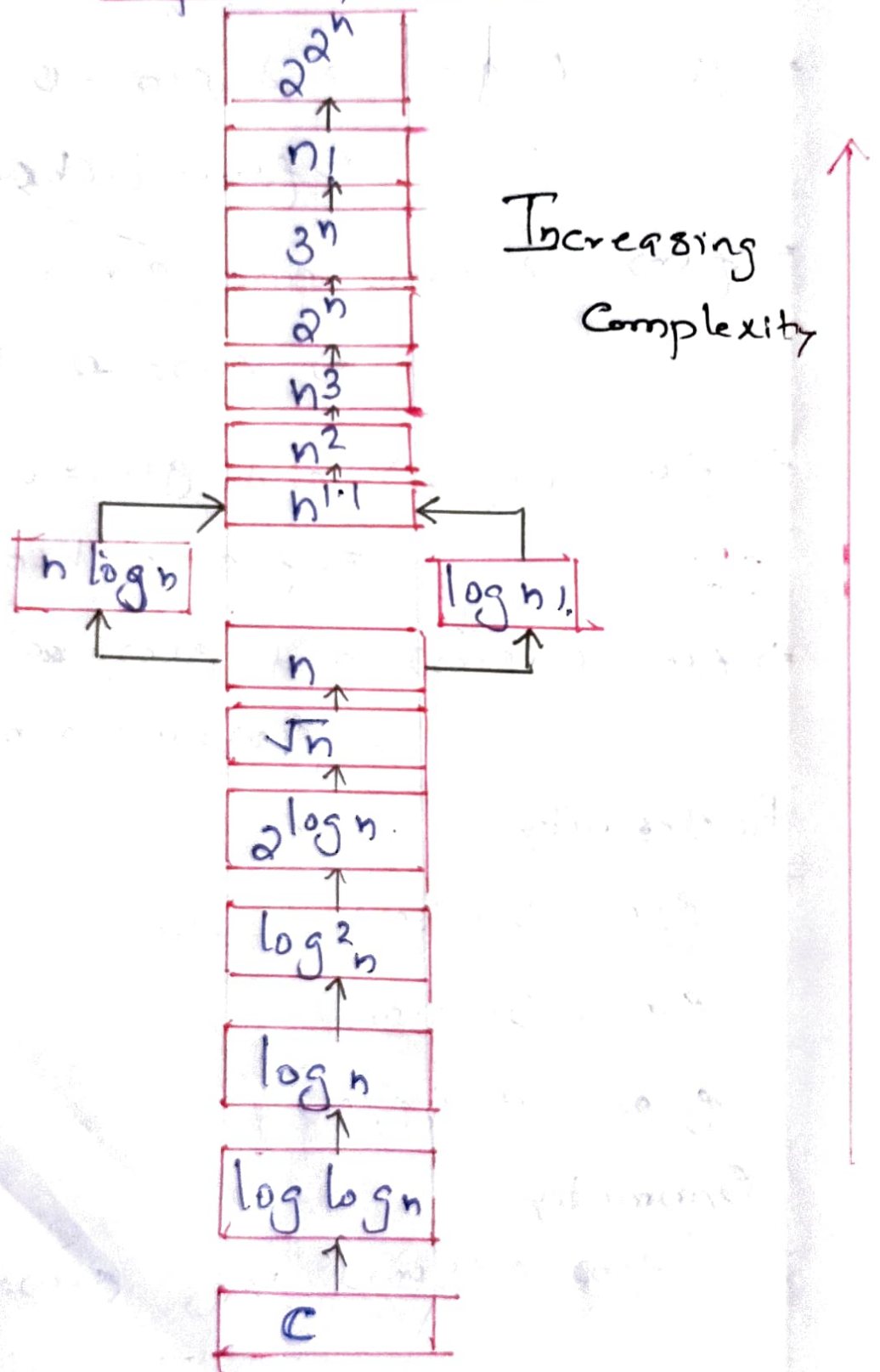
$$f(n) = O(g(n)) \text{ iff } g(n) = O(f(n))$$

Transpose Symmetry.

$$f(n) = O(g(n)) \iff g(n) = -\Omega(f(n))$$

$$f(n) = \Theta(g(n)) \iff g(n) = \omega(f(n))$$

Complexity chart.



Algorithm Analysis - classifying functions

The main object of analysis is to represent the rate of growth of time of algm. as a function of input size by means of mathematical functions

These functions can come in the form of either polynomials or exponentials.

Under the polynomial we can have

⇒ Constants, $[c]$

⇒ logarithmic, $[\log n]$

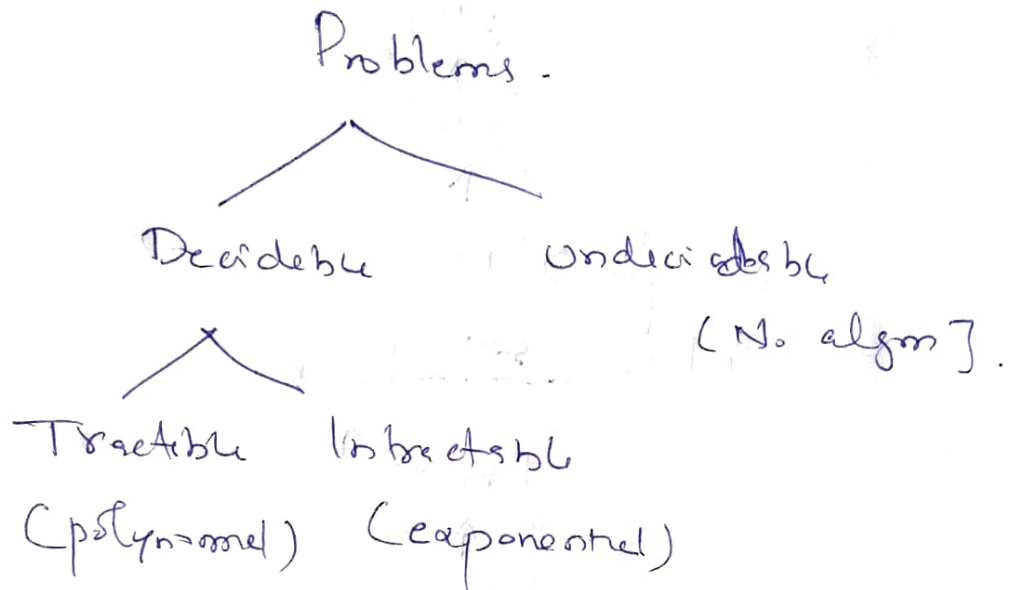
⇒ Linear $[n]$

⇒ Quadratic $[n^2]$

⇒ Cubic $[n^3]$

Exponentials include 2^n , 4^n , etc.

$\Rightarrow$ In general; it is desirable to develop algorithms for problems having polynomial complexity.



Complexity.

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < n^n.$$

or.

$$c < \log n < \log^2 n < \sqrt{n} < n < n \log n < n^{1.1} < n^2 < n^3 < 2^n < 3^n < n! < 2^{2^n}$$